

Memory Layouts and Allocation (Stack & Heap)

Given the memory layout illustrated below *fig 1.1*, each memory location is pointed with address which is represented in base 16 number representation and it goes from 0 all the way to N from bottom to top.

Within this memory layout the first row represents stack memory allocation which grows towards bottom which means it starts from 0xN... to next address ($\text{address} = \text{stack pointer} - 1$) in a contiguous order operating in LIFO principle by popping the last pushed memory address from the bottom as illustrated in *fig 1.2*.

The middle row represents the Heap memory which sits on top of data memory allocation and text memory allocation and it grows towards top from its starting base address as illustrated in *fig 1.3*.

Data allocation is for all global variables which stores initialised and uninitialised data separately. Data allocated memory slot does not grows and remains **fixed size**.

Text allocation in memory stores all compiled machine code (instructions) as raw text like:
00100010 00100010 00100010 00100010 00100010 00100010 00100010 00100010 00100010 00100010
Text allocated memory slots are **fixed size** since the compiled binaries of a program does not grows or decrease when in execution.

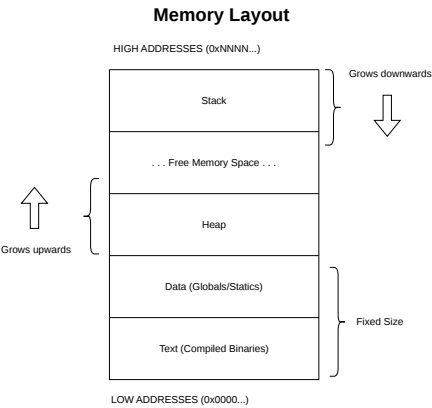


fig 1.1 High Level Memory Layout

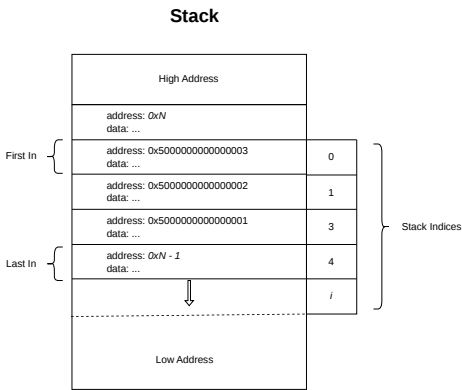


fig 1.2 Stack memory layout

starts from higher address and grows contiguously towards low address in LIFO order, which means the last address at index 4 will be the first to get accessed

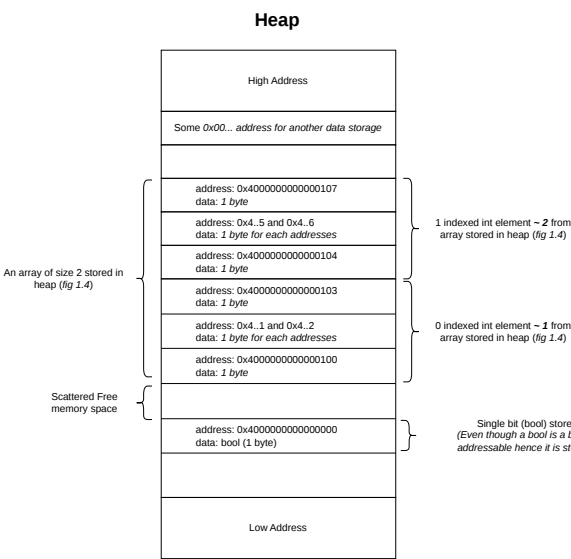


fig 1.3 Heap memory layout

starts from low address on top data allocated address and grows in a scattered manner towards higher addresses

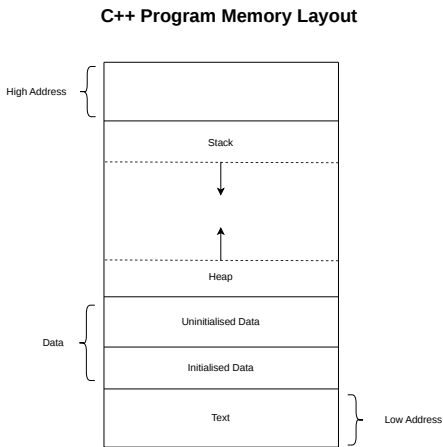


fig 2.1 Memory layouts of a C++ program

Illustration from CS 225 | Stack and Heap Memory, University of Illinois Urbana-Champaign

